

Raport ze wstępnych badań projektu zespołowego Big Data

Propozycja trasy na podstawie otwartych danych nawigacyjnych dla miasta stołecznego Warszawy

Zespół 12:

Michał Zalewski

Jakub Próchnicki

Jan Machowski

Maciej Kurzydłowski

Jan Domański

Warszawa, 21 maja 2026

Spis treści

1	Wstęp	3
2	Infrastruktura oraz potok przetwarzania danych	3
2.1	Przedstawienie infrastruktury	3
2.2	Potok danych	4
2.2.1	Źródła danych	4
2.2.2	Pobór danych - Kafka	7
2.2.3	Czyszczenie i transformacja danych - Apache Spark, OSMnx	8
2.2.4	Zapis danych	9
3	Instalacja, konfiguracja oraz wykonanie projektu	9
3.1	Instalacja i zbudowanie projektu	9
3.2	Pobór danych (ingestion)	9
3.3	Konfiguracja skryptów analizy	10
3.4	Dane wyjściowe	11
4	Wstępna analiza problemu	11
4.1	Cel i metodyka	11
4.2	Wyniki wstępne	11
4.3	Wnioski	12
5	Dalsze kierunki badania	13
6	Podsumowanie	13

1 Wstęp

Niniejszy raport jest kontynuacją prac dążących do opracowania modelu propozycji trasy na podstawie otwartych danych nawigacyjnych. Główny cel tego dokumentu to przedstawienie stworzonej w ramach projektu infrastruktury oraz potoku przetwarzania danych. Opisana zostanie również wstępna analiza wyżej wymienionego problemu oraz wnioski wynikające z tej analizy.

W poprzednim dokumencie zawarty został przegląd tematyki, wstępna propozycja źródeł oraz narzędzi wykorzystanych do wykonania projektu, oraz problemów i ograniczeń związanych z wybranymi danymi.

Dla przypomnienia - zaproponowana została reprezentacja miasta w postaci grafu skierowanego, gdzie wierzchołki są przecięciami lub zakończeniami dróg a krawędzie są faktycznymi odcinkami dróg, z odpowiednim zestawem atrybutów - długość, typ drogi, liczba pasów, natężenie ruchu etc. Propozycja ta została wykorzystana do stworzenia wstępnego rozwiązania, które zostanie przedstawione poniżej.

2 Infrastruktura oraz potok przetwarzania danych

2.1 Przedstawienie infrastruktury

Projekt został przygotowany jako środowisko kontenerowe - uruchamiany jest za pomocą **Docker Compose**. Pozwala to na odtworzenie środowiska na różnych urządzeniach, nie musząc instalować ręcznie zależności systemowych / bibliotek. W kontenerze **opennavigation** znajdują się narzędzia potrzebne do pobierania, przetwarzania i analizy danych.

Język **Python** został wybrany jako główny język implementacji. Jednakże **OSMnx** i **NetworkX** są wykorzystywane do pracy z grafem drogowym Warszawy.

Pandas służy w projekcie do przetwarzania danych tabelarycznych, agregacji wyników, zapisu plików wynikowych.

W ramach kontenera zainstalowane są także narzędzia **Apache Kafka** oraz **Apache Spark**. To pierwsze jest systemem wymiany danych, który umożliwia przesyłanie danych między różnymi etapami potoku przetwarzania. Apache Spark, natomiast, umożliwia przetwarzanie danych strumieniowych i transformację danych surowych w formę, która jest już gotowa do analizy.

W projekcie nie wykorzystano Hadoopa, co jest zgodne z przyjętym zakresem projektu (był traktowany jako opcjonalne narzędzie). Zamiast tego dane są przetwarzane lokalnie, a wyniki są zapisywane w strukturze katalogów projektu.

Struktura danych w projekcie podzielona jest na trzy katalogi:

- **data/raw** - przechowuje dane surowe, m. in. raporty XLSX/PDF zaczerpnięte ze źródeł zewnętrznych
- **data/processed** - przechowuje dane oczyszczone oraz zagregowane, które wykorzystywane są już prosto przez analizę tras
- **data/results** - przechowuje wyniki analiz takie jak porównania tras

W obecnej wersji projektu, możliwe jest uruchomienie pełnej analizy trasy:

1. Pobranie danych źródłowych (np. raporty XLSX/PDF z zewnętrznych źródeł)
2. Przetworzenie danych (czyszczenie, transformacja, agregacja)
3. Wyznaczenie trasy na podstawie przetworzonych danych
4. Porównanie różnych wariantów trasy według różnych funkcji kosztu

2.2 Potok danych

Potok danych został przygotowany w takim sposób, aby połączyć różne źródła danych w jeden spójny model kosztu przejazdu.

Podstawą analizy jest graf drogowy Warszawy - stworzony na podstawie danych z **OpenStreet-Map** z pomocą biblioteki **OSMnx**. Graf ten jest następnie wzbogacany o dane:

- **ZDM** - dane dotyczące natężenia ruchu oraz średnich prędkości pojazdów (dane te są jedynie punktowe, więc są wykorzystywane do korekty kosztu przejazdu tylko dla tych krawędzi, dla których są dostępne)
- **Dane pogodowe** - dane dotyczące warunków pogodowych, które mogą wpływać na czas przejazdu, np. deszcz czy mgła

Potok obejmuje następujące etapy:

- Pobór danych
- Czyszczenie i transformację danych
- Agregację danych do postaci użytecznej w analizie tras
- Zapis wyników pośrednich i końcowych

Szczegółowy opis poszczególnych etapów potoku przedstawiono w kolejnych podpunktach.

2.2.1 Źródła danych

Źródła danych obejmują zarówno statyczny opis sieci drogowej, jak i dane zmienne w czasie. W projekcie wykorzystano następujące źródła:

- **OpenStreetMap / Overpass API** – geometria sieci drogowej Warszawy oraz atrybuty odcinków (np. *highway*, *maxspeed*, *lanes*, *oneway*). Te dane są podstawą do budowy grafu tras i wyliczenia bazowych czasów przejazdu. Przykład zapytania Overpass API używanego w projekcie (wybór dróg głównych dla Warszawy):

```
[out:json][timeout:120];
area["name"="Warszawa"]->.a;
(
  way["highway"~"^(primary|secondary|tertiary)$"](area.a);
);
(._;>);
out body;
```

Przykładowy fragment odpowiedzi (rzeczywisty wynik, ograniczony do pojedynczego odcinka dla czytelności):

```
{
  "elements": [
    {
      "type": "way",
      "id": 4307329,
      "nodes": [
        2448759046,
        7093785352,
        2452307268,
        1439574696
      ],
      "tags": {
        "cycleway:both": "no",
        "highway": "tertiary",
        "junction": "roundabout",
        "lane_markings": "no",
        "lit": "yes",
        "maxspeed": "50",
        "name": "Rondo Feliksa Stamma",
        "name:etymology:wikidata": "Q5441838",
        "sidewalk:left": "no",
        "sidewalk:right": "separate",
        "source:maxspeed": "PL:urban",
        "surface": "asphalt",
        "wikidata": "Q113528575"
      }
    }
  ]
}
```

- **ZDM Warszawa (APR)** – punktowe pomiary natężenia ruchu na wybranych odcinkach. Dane są pobierane z panelu *Analiza ruchu na drogach online* (<https://zdm.waw.pl/dzialania/badania-i-analizy/analiza-ruchu-na-drogach/analiza-ruchu-na-drogach-online/>) w postaci raportu XLSX. Pobranie jest zautomatyzowane skryptem `zdm_online_traffic_loader.py`, który steruje interfejsem WWW (Playwright) i zapisuje raport do pliku.

Tabela 1 przedstawia uproszczony format rekordu APR, który trafia do potoku po wczytaniu raportu XLSX.

Tabela 1: Format danych ZDM APR (po wczytaniu do potoku)

Pole	Opis
source	Identyfikator źródła (np. <i>zdm_apr_real</i>).
station_id	Id punktu pomiarowego ZDM.
station_name	Nazwa punktu pomiarowego (np. ulica/sekcja).
road_name	Nazwa ulicy.
section	Opis odcinka (np. między skrzyżowaniami).
direction	Kierunek pomiaru.
lat, lon	Współrzędne geograficzne punktu.
measurement_time	Czas pomiaru w formacie <i>YYYY-MM-DD HH:00:00</i> .
period_minutes	Okno czasowe pomiaru (zwykle 60).
vehicle_count	Liczba pojazdów w danym oknie.
avg_speed_kmh	Brak w APR (pole puste).
_metadata	Metadane techniczne: czas wczytania, plik źródłowy, arkusz.

- **Raporty prędkości ZDM (PDF)** – cykliczne raporty z uśrednionymi prędkościami, publikowane na dashboardie ZDM (<https://zdm-warszawa.maps.arcgis.com/apps/dashboards/20bffffc9bae544b5a1cdfaad7be38c40>). Pobranie plików jest zautomatyzowane skryptem `zdm_speed_report_downloader.py`. Następnie skrypt `zdm_speed_pdf_parser.py` parsuje PDF-y do CSV (`zdm_speed_measurements.csv`).

Tabela 2 pokazuje uproszczony format rekordu z raportów prędkości.

Tabela 2: Format danych ZDM – raporty prędkości (po parsowaniu PDF)

Pole	Opis
source	Identyfikator źródła (np. <i>zdm_speed.pdf</i>).
source_file	Nazwa pliku PDF, z którego pochodzi rekord.
station_id	Id punktu pomiarowego ZDM.
station_name	Nazwa punktu (np. ulica/sekcja).
road_name	Nazwa ulicy.
direction	Kierunek pomiaru.
lat, lon	Współrzędne geograficzne punktu.
measurement_date	Data raportu (YYYY-MM-DD).
measurement_time	Znaczniki czasu (domyślnie 00:00:00).
period_minutes	Okres uśrednienia (1440 minut).
day_of_week	Dzień tygodnia z raportu.
vehicle_count	Liczba pojazdów (jeśli dostępna).
avg_speed_kmh	Średnia prędkość dla kierunku/dnia.
avg_speed_day_kmh	Średnia prędkość w dzień.
avg_speed_night_kmh	Średnia prędkość w nocy.
avg_speed_light_kmh	Prędkość dla pojazdów lekkich.
avg_speed_heavy_kmh	Prędkość dla pojazdów ciężkich.
vmax_kmh, v85_kmh	Prędkości Vmax i V85 (jeśli dostępne).
vdop_raw	Dopuszczalna prędkość (surowy zapis z PDF).

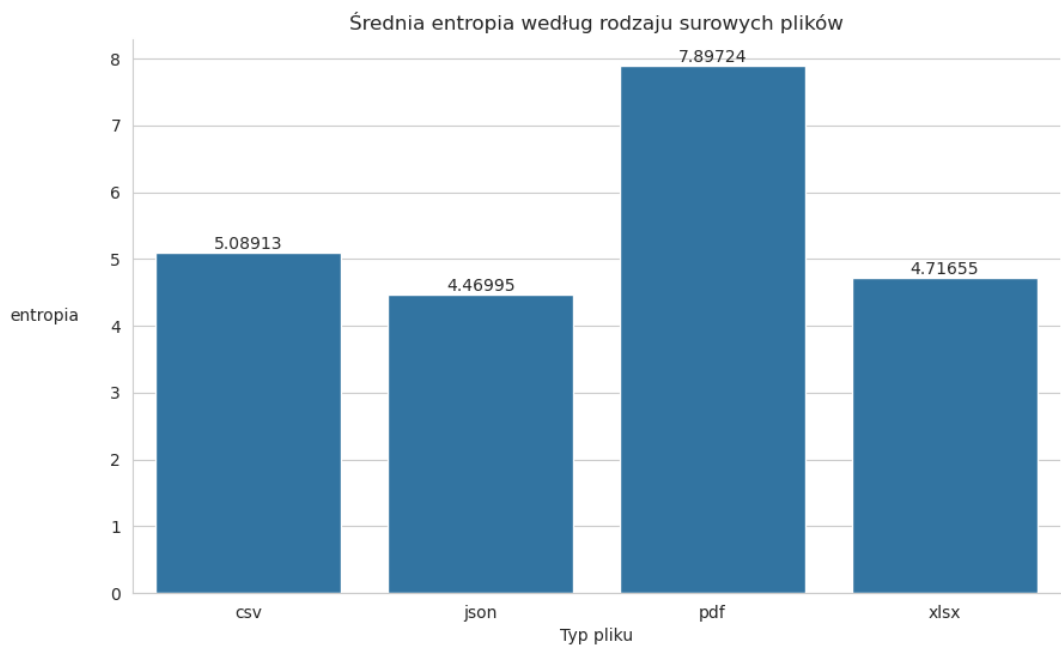
- **Open-Meteo** – dane pogodowe godzinowe (opady, wiatr, widoczność, temperatura), pobierane z API. Pobranie realizuje skrypt `weather_loader.py`, który zapisuje pełny JSON do Kafki (temat *warszawa-raw-weather*). Dane te stanowią kontekst do dodatkowej korekty kosztu.

Przykład zapytania API (parametry używane w projekcie):

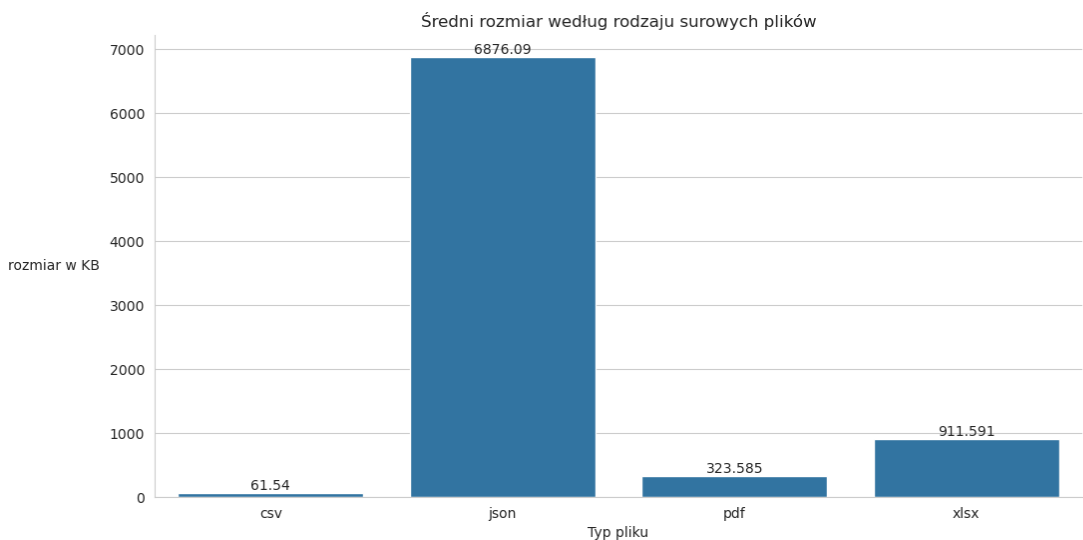
```
https://api.open-meteo.com/v1/forecast?
latitude=52.2297&
longitude=21.0122&
hourly=temperature_2m,precipitation,rain,snowfall,wind_speed_10m,
visibility,weather_code&
forecast_days=1&
timezone=Europe/Warsaw
```

Przykładowy fragment odpowiedzi (rzeczywisty wynik, skrócony do kilku godzin):

```
{
  "latitude": 52.23009,
  "longitude": 21.017075,
  "timezone": "Europe/Warsaw",
  "hourly": {
    "time": [
      "2026-05-22T00:00",
      "2026-05-22T01:00",
      "2026-05-22T02:00"
    ],
    "temperature_2m": [16.9, 15.9, 14.9],
    "precipitation": [0.0, 0.0, 0.0],
    "wind_speed_10m": [11.2, 8.6, 6.1],
    "weather_code": [3, 0, 3]
  }
}
```



Rysunek 1: Średnia entropia, rozumiana w sensie teorii informacji, dla poszczególnych typów plików źródłowych (nieprzetworzonych)



Rysunek 2: Średni rozmiar pliku źródłowego według typu pliku źródłowego (nieprzetworzonego)

Pliki typu pdf są najmniej ustrukturyzowanym źródłem danych w naszym projekcie, co potwierdza wysoka entropia zawartości tych plików. Największy udział w danych źródłowych pod względem rozmiaru mają pliki w formacie JSON, które jednocześnie charakteryzuje najmniejsza entropia, co może oznaczać najwyższy poziom ustrukturyzowania spośród wszystkich wykorzystywanych typów plików.

2.2.2 Pobór danych - Kafka

Pobór danych realizowany jest przez skrypty loaderów, które zapisują dane wejściowe jako wiadomości JSON w tematach Apache Kafka. Poniżej zestawiono, co Kafka przyjmuje, do jakich tematów oraz gdzie dane są dalej przekazywane:

- **warszawa-raw-streets** – dane z Overpass API; producent `overpass_raw_street_loader.py`; konsument Spark `overpass_transformations.py`; zapis do `/app/data/processed/osm_ways` oraz publikacja do tematu `warszawa-processed-edges`.
- **warszawa-raw-traffic** – dane APR z ZDM; producenci `zdm_traffic_loader.py` lub `traffic_loader.py`; konsument `traffic_transformations.py`; zapis do `/app/data/processed/traffic_measurements`, a następnie agregacja `zdm_traffic_aggregation.py` do `/app/data/processed/zdm_traffic_by_road.csv`.

- **warszawa-raw-weather** – dane Open-Meteo; producent `weather_loader.py`; konsument `weather_transformations.py`; zapis do `/app/data/processed/weather_observations`.
- **warszawa-processed-edges** – ujednolicone krawędzie grafu po transformacji OSM; temat wyjściowy z `overpass_transformations.py`, możliwy do dalszego wykorzystania analitycznego.

W obecnej wersji zasilanie odbywa się wsadowo (mikro-batche), jednak potok działa w trybie streamingowym po stronie Apache Spark. Kafka pełni rolę bufora i kolejki zdarzeń, co zwiększa powtarzalność obliczeń i pozwala na odtwarzanie danych od wybranego offsetu.

2.2.3 Czyszczenie i transformacja danych - Apache Spark, OSMnx

Ten etap ma na celu przekształcenie danych w formie surowej do takiej postaci, która jest już bezpośrednio użyteczna w analizie tras.

Dane **ZDM APR** są pobierane do Apache Kafka w surowej postaci, a następnie przetwarzane z pomocą Apache Spark. Spark odczytuje komunikaty z **warszawa-raw-traffic** i interpretuje je jako rekordy JSON. Ostatecznie przekształca je do postaci tabelarycznej - nazwy kolumn są ujednolicone, czas pomiaru jest konwertowany do formatu daty i czasu a liczba pojazdów jest przeliczana na natężenie ruchu (pojazdy/h). Wyznaczany jest właśnie na tym etapie współczynnik **traffic_factor**, który jest używany jako **kara kosztu przejazdu** dla tych ulic, które mają dostępne pomiary natężenia ruchu.

Dane **ZDM APR** są agregowane po ulicach. Z wielu rekordów godzinowych wybierany jest m. in. maksymalny/średni **traffic_factor**, liczba pomiarów oraz zakres dat. Wynik tego to plik **zdm_traffic_by_road.csv** - pozwala on na uruchomienie analizy tras bez ponownej potrzeby przetwarzania dużych danych wejściowych.

W kwestii danych o prędkości pobieranych z raportów prędkości ZDM (w formie PDF), proces jest bardziej złożony. Dane są przetwarzane osobnym parserem. Pobierane są zawartości plików PDF, a później wydobywa się z nich kluczowe informacje takie jak:

- Identyfikator punktu pomiarowego
- Nazwa ulicy
- Lokalizacja (współrzędne geo.)
- Data pomiaru
- Średnia prędkość **avg_speed_kmh**

Zdarza się, że dla jednej konkretnej ulicy istnieje kilka pomiarów z różnych okresów czasu. Wtedy wylicza się wartość zagregowaną, która może być średnią arytmetyczną lub średnią ważoną liczbą pojazdów, jeśli ta informacja jest dostępna. Wynikiem tego etapu jest plik **zdm_speed_by_road.csv**.

Za pomocą biblioteki **OSMnx** powstaje struktura sieci drogowej Warszawy. Dla danego obszaru miasta pobiera się graf drogowy typu **drive**, który to odpowiada samochodowym pojazdom. Automatycznie wyznaczane są bazowe prędkości/czasu przejazdu dla krawędzi. W następnej kolejności krawędzie są wzbogacane o atrybuty:

- **traffic_factor** - korekta kosztu przejazdu na podstawie ZDM APR (o natężeniu ruchu)
- **speed_factor** - korekta na podstawie prędkości ruchu z raportów ZDM
- **weather_factor** - korekta na podstawie danych o pogodzie
- **traffic_speed_factor** - finalna korekta wynikająca z danych ruchu i prędkości - jest ona wyznaczana jako większa z wartości **traffic_factor** i **speed_factor**
- **cost_time_s** - końcowa waga krawędzi w grafie (czas w sekundach), uwzględniająca bazowy czas przejazdu i dostępne korekty

Na etapie transformacji normalizowane są też nazwy ulic, bowiem ZDM oraz OpenStreetMap mogą mieć różne formy zapisu nazw konkretnych ulic, np. Al. i Aleje. Potok stosuje uproszczone ujednolicanie nazw i usuwanie wybranych skrótów. Jest to istotne, ponieważ dane ZDM są głównie dopasowywane do krawędzi po nazwach ulic, co na późniejszym etapie zostanie zastąpione dopasowaniem przestrzennym.

2.2.4 Zapis danych

Kiedy dane są już oczyszczone, przekształcone i wzbogacone o dodatkowe atrybuty, są następnie zapisywane do odpowiednich katalogów w projekcie. W tej kwestii zastosowano następujący podział:

- Dane surowe (np. raporty XLSX/PDF) są zapisywane w katalogu **data/raw**
- Dane przetworzone, takie jak pomiary natężenia ruchu czy dane pogodowe, są zapisywane w katalogu **data/processed**. Format zapisu dla danych tabelarycznych to Parquet.
- Wyniki analiz są zapisywane w katalogu **data/results**.

3 Instalacja, konfiguracja oraz wykonanie projektu

3.1 Instalacja i zbudowanie projektu

Aktualną wersję rozwiązania można znaleźć w otwartym repozytorium na GitHub, pod linkiem <https://github.com/JacobPW23/OpenNavigation>.

Aby zapewnić uniwersalność rozwiązania i prostotę obsługi i udostępniania postanowiono skonteneryzować projekt za pomocą technologii Docker. W celu wykorzystania OpenNavigation należy sklonować repozytorium, zbudować obraz oraz uruchomić kontener na podstawie zbudowanego obrazu:

```
> git clone https://github.com/JacobPW23/OpenNavigation
> cd OpenNavigation
> docker compose build
> docker compose up -d
```

Należy zwrócić uwagę, iż z powodu instalowania narzędzi Apache, zbudowanie obrazu Docker'a może chwilę zająć.

3.2 Pobór danych (ingestion)

Pobieranie danych jest zautomatyzowane za pomocą skryptów Pythona, które można uruchomić w kontenerze Docker. Poniżej przedstawiono komendy do pobrania danych z poszczególnych źródeł:

Dane APR ZDM

```
> docker compose exec opennavigation python \
    src/ingestion/zdm_online_traffic_loader.py
> docker compose exec opennavigation python \
    src/ingestion/zdm_traffic_loader.py
> docker compose exec opennavigation spark-submit \
    --packages \
    org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 \
    src/processing/traffic_transformations.py
> docker compose exec opennavigation python \
    src/analysis/zdm_traffic_aggregation.py
```

Wyniki: data/processed/zdm_traffic_by_road.csv.

Raporty prędkości ZDM (PDF)

```
> docker compose exec opennavigation python \
    src/ingestion/zdm_speed_report_downloader.py
> docker compose exec opennavigation python \
    src/ingestion/zdm_speed_pdf_parser.py
> docker compose exec opennavigation python \
    src/analysis/zdm_speed_aggregation.py
```

Wyniki: data/processed/zdm_speed_by_road.csv.

OSM / Overpass

```
> docker compose exec opennavigation python \
    src/ingestion/overpass_raw_street_loader.py
> docker compose exec opennavigation spark-submit \
    --packages \
    org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 \
    src/processing/overpass_transformations.py
```

Wyniki: temat *warszawa-processed-edges*.

Pogoda (Open-Meteo)

```
> docker compose exec opennavigation python \
    src/ingestion/weather_loader.py
> docker compose exec opennavigation spark-submit \
    --packages \
    org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 \
    src/processing/weather_transformations.py
```

Wyniki: *warszawa-raw-weather*.

3.3 Konfiguracja skryptów analizy

Po udanym postawieniu kontenera, i sprawdzeniu jego działania za pomocą `docker compose ps`, dane mogą zostać pobrane według potoku danych.

Skrypty wykonujący analizę najoptymalniejszej trasy na podstawie danych znajduje się pod następującymi ścieżkami:

- /app/src/analysis/route_initial_analysis.py
- /app/src/analysis/osm_initial_analysis.py

W tym raporcie opisane zostanie wykorzystanie pierwszego skryptu. Pozostałe zostaną opisane w ramach raportu końcowego.

Kluczowym punktem dla analizy jest zdefiniowana w tym pliku stała **ROUTES**, w której określone są punkty początkowe i końcowe, dla których ma zostać znaleziona optymalna trasa. Poniżej zamieszczono przykładową konfigurację zmiennej, domyślnie znajdującą się w skrypcie w języku Python:

```
ROUTES = [  
    {  
        "route_name": "Mokotowska -> Dobra",  
        "origin": (52.218662, 21.017080),  
        "destination": (52.244974, 21.020670),  
    },  
]
```

gdzie współrzędne *origin* oraz *destination* są podawane jako (*latitude*, *longitude*).

Aby uruchomić analizę tras, należy wykonać plik analizy tras po wprowadzeniu swojej konfiguracji. Komenda:

```
> docker compose exec opennavigation python \  
    src/analysis/route_initial_analysis.py
```

3.4 Dane wyjściowe

Wyniki wykonania skryptu znajdują się w katalogu `/app/data/results/route_initial_analysis/`. Dla analizy danych istotne są 3 pliki wynikowe:

1. `summary.md` - podsumowanie wyniku analizy skryptu;
2. `routes_comparison.csv` - plik csv podsumowujący najważniejsze aspekty determinujące proponowane trasy: `distance_km`, `base_time_min`, `adjusted_time_min`, `traffic_impacted_edges`, `speed_impacted_edges`, `zdm_apr_edges`, `zdm_speed_edges`, `osmnx_only_edges`;
3. `route_edges.csv` - szczegóły krawędzi najbardziej optymalnej trasy znalezionej przez algorytm, takie jak `traffic_factor`, `speed_factor`, `traffic_speed_factor`, `measured_avg_speed_kmh`, `cost_source`.

4 Wstępna analiza problemu

4.1 Cel i metodyka

Celem rozdziału jest weryfikacja, czy algorytm znajduje sensowną trasę oraz czy czas przejazdu w minutach jest liczony spójnie. Dodatkowo wyniki porównano z czasami i dystansami z Google Maps dla aktualnego obciążenia ruchu (19:00–19:30), aby ocenić zgodność kierunkową. Analiza opiera się na wynikach generowanych przez `route_initial_analysis.py` i zapisanych w `data/results/route_initial_analysis`.

Przyjęta metodyka:

1. Wybrano trzy pary punktów start–cel (*ROUTES*).
2. Dla każdej pary wyznaczono trzy warianty: najkrótszy dystansowo, najszybszy bazowo (na podstawie *travel_time* z OSMnx) oraz wariant skorygowany (*cost_time_s* z uwzględnieniem danych ZDM i pogody).
3. Dla każdej trasy zsumowano długość oraz czasy przejazdu, uzyskując *distance_km*, *base_time_min* i *adjusted_time_min*.
4. Wygenerowane trasy przepisano do Google Maps i odczytano czas oraz dystans dla okna 19:00–19:30.

4.2 Wyniki wstępne

Najpierw zestawiono wyniki generowane przez program (na podstawie *summary.md*) w tabeli 3.

Tabela 3: Wyniki programu dla trzech tras

Trasa	Wariant	Dystans [km]	Czas bazowy [min]	Czas skoryg. [min]
Mokotowska → Dobra	najkrótsza dystansowo	4.73	5.99	7.09
Mokotowska → Dobra	najszybsza bazowo	4.78	5.80	6.56
Mokotowska → Dobra	skorygowana ruchem i pogodą	4.78	5.80	6.56
Dworzec Centralny → Stadion Narodowy	najkrótsza dystansowo	4.32	5.20	6.90
Dworzec Centralny → Stadion Narodowy	najszybsza bazowo	4.32	5.20	6.90
Dworzec Centralny → Stadion Narodowy	skorygowana ruchem i pogodą	4.97	6.35	6.69
Ochota → Powiśle	najkrótsza dystansowo	6.78	8.32	11.14
Ochota → Powiśle	najszybsza bazowo	6.79	8.20	10.94
Ochota → Powiśle	skorygowana ruchem i pogodą	7.14	8.84	9.48

- Wszystkie przedstawione algorytmy znalazły poprawne trasy, na podstawie ich miar. *Najkrótszy dystansowo* ma najkrótszy dystans, *najszybszy bazowo* ma najkrótszy czas bazowy, a *skorygowany* najkrótszy czas skorygowany.

Wyniki porównawcze przygotowano na podstawie ręcznego przepisania wygenerowanych tras do Google Maps i odczytu czasu oraz dystansu dla aktualnego ruchu w godzinach 19:00–19:30. W tabeli podano wartości w formacie *min / km*.

Tabela 4: Porównanie czasu i dystansu z Google Maps (19:00–19:30)

Trasa	Google [min/km]	Najkrótsza [min/km]	Bazowa [min/km]	Skorygowana [min/km]
Mokotowska → Dobra	19 / 6.5	22 / 4.6	26 / 4.9	26 / 4.9
Dworzec Centralny → Stadion Narodowy	34 / 4.5	34 / 4.5	34 / 4.5	30 / 5.2
Ochota → Powiśle	24 / 8.8	34 / 7.5	30 / 7.0	29 / 7.0

- Dla trasy Mokotowska–Dobra czasy z programu są wyższe niż w Google Maps, mimo krótszego dystansu. Jest to spowodowane korkami nie przewidzianymi przez dane o zagęszczeniu trasy. Wariant bazowy i skorygowany nie polepszyły czasu względem najkrótszej trasy, co sugeruje, że dobrane współczynniki bądź niestandardowym ruchem w tej części miasta.
- Dla trasy Dworzec Centralny–Stadion Narodowy wariant najkrótszy i bazowy dały pokrywają się z czasem z Google Maps. Wariant skorygowany poprawił czas, co sugeruje, że korekta na podstawie danych ZDM i pogody była trafna i poprawiła wynik względem automatycznej trasy Google Maps.
- Dla trasy Ochota–Powiśle algorytm źle wyznaczył trasę, co skutkowało większym dystansem dla najkrótszej trasy niż w wariancie bazowym oraz skorygowanym. Ten błąd jest spowodowany zapisem grafowym OSM, gdzie nie ma oznaczonych zakazów skrętu oraz zamkniętych ulic, które uwzględnia Google Maps. Wariant skorygowany minimalnie poprawił wynik względem bazowego.

4.3 Wnioski

Na podstawie porównań można stwierdzić, że algorytm konsekwentnie wyznacza trzy warianty tras i poprawnie różnicuje je według przyjętych miar (dystans, czas bazowy, czas skorygowany). Korekta kosztu na podstawie danych ZDM i pogody potrafi zmienić wybór trasy i obniżyć czas przejazdu względem wariantu bazowego, co potwierdza sens podejścia. Różnice względem Google Maps wskazują na potrzebę dostosowania współczynników oraz uzupełnienia ograniczeń w grafie OSM (np. zakazy skrętu, zamknięcia).

Algorytm skorygowany radzi sobie lepiej niż bazowy oraz najkrótszy, jednak nadal nie obsługuje ruchu w czasie rzeczywistym, co oznacza że nie powinien być używany do omijania zatorów w czasie rzeczywistym, a raczej do planowania trasy z uwzględnieniem danych historycznych i kontekstowych.

5 Dalsze kierunki badania

W kolejnych etapach powinniśmy skupić się na poprawieniu znajdowania najszybszej trasy oraz poprawie szacowania czasu przejazdu. W tym celu należy rozważyć następujące kierunki:

- **Kalibracja współczynników** – zmiana *traffic_factor* oraz *speed_factor* aby poprawić trafność wyznaczanych tras.
- **Ograniczenia drogowe** – uzupełnienie grafu OSM o zakazy skrętu, zamknięcia i ograniczenia kierunkowe, aby ograniczyć rozbieżności z rzeczywistymi trasami.
- **Dane czasu rzeczywistego** – dodanie często aktualizowanych źródeł (np. zdarzenia drogowe, zagęszczenia ruchu).
- **Walidacja zewnętrzna** – porównanie wyników z rzeczywistymi czasami przejazdu z łatwo dostępnych źródeł (Google Maps nie daje możliwości łatwego zautomatyzowania tworzenia własnych tras)

6 Podsumowanie

W raporcie przedstawiono zbudowaną infrastrukturę projektu oraz zarys potoku przetwarzania danych, który obejmuje pobór danych, buforowanie w Kafka, przetwarzanie w Spark i zapis wyników do struktury katalogów projektu. Opisano kluczowe źródła danych wraz z formatami i przykładowymi wynikami zapytań, co stanowi podstawę do budowy grafu dróg i korekty kosztów przejazdu.

W części analitycznej występuje wstępne porównania wariantów tras dla kilku par punktów, zestawiając wyniki programu z danymi z Google Maps. Analiza pokazuje, że algorytmy poprawnie generują warianty tras pod kątem dystansu i czasu, a korekty na podstawie danych ZDM i pogody potrafią poprawić wynik, choć wciąż występują rozbieżności wynikające z ograniczeń danych oraz uproszczeń w grafie OSM.

Zidentyfikowane ograniczenia wskazują na potrzebę dalszej kalibracji współczynników i uzupełnienia ograniczeń drogowych oraz rozbudowy źródeł danych. Zaproponowane kierunki badań koncentrują się na poprawie trafności wyznaczania tras i zbliżeniu wyników do rzeczywistych czasów przejazdu.